Основы теории баз данных

Содержание

1. Первые представления о базах данных
    1. Понятие данных
    2. Архитектура данных
        1. Уровни архитектуры
        2. Модель «сущность — связь»
        3. Логические модели данных
2. Реляционные базы данных
    1. Ключевые понятия
        1. Отношение
        2. Ключи
        3. Реляционная алгебра
    2. Выкладки из SQL
    3. Нормализация

”Вопрос был не в том, что хуже или лучше: это всё было равномерно плохо.”
— Маятин А. В.

Первые представления о базах данных

Понятие данных

Что такое информация?

Ёмкого ответа на этот вопрос не существует. В зависимости от того, с какой стороны мы к нему подходим, информацию можно определять совершенно по-разному. В то же время сама она вполне объективна, и нам не обязательно её понимать для того, чтобы изучать её свойства.

Данные можно представить как одну из форм информации наряду с сигналами или знаниями. Стандарт ISO/IEC 2382:2015 (или, если угодно, ГОСТ 33707-2016) определяет их следующим образом:

ⓘ Определение

Данные — поддающееся многократной интерпретации представление информации в формализованном виде, пригодном для передачи, интерпретации или обработки.

Реинтерпретируемость здесь подразумевает, что данные статичны и каждый раз представляются в одном и том же виде, в отличие, например, от знаний.

Уже в самòм определении подчёркнута ключевая роль *формализации* данных, поскольку именно она позволяет производить над ними хоть какие-то содержательные действия. Как информация, так и данные могут иметь весьма внушительный объем, в связи с чем может возникнуть желание упростить работу с ними через *структурирование*, которое также невозможно без формализации. Можно зайти дальше и сказать, что раз объём данных велик, а их вид позволяет нам обрабатывать их каким угодно образом, то естественным желанием будет делегировать работу с ними компьютерным системам.

Из этих соображений во второй половине предыдущего столетия возникает понятие *базы данных*, общепризнанная формулировка которого вновь отсутствует, а существующие отражают скорее субъективное восприятие авторов. Согласно всё тому же ISO/IEC 2382:2015,

ⓘ Определение

База данных — совокупность данных, организованных в соответствии с концептуальной структурой, описывающей характеристики этих данных и взаимоотношения между ними, которая поддерживает одну или более областей применения.

Это довольно широкий взгляд на вопрос, согласно которому базой данных является, например, библиотека. В этом нет концептуального противоречия, однако обычно под термином «база данных» всё-таки подразумевается более узкое явление, основной чертой которого является уже упомянутая ориентированность на вычислительные (или, говоря ещё точнее, электронно-вычислительные) системы. Именно в таком контексте будет вестись разговор о них в дальнейшем.

Базы данных по обыкновению сопровождаются *системами управления*, которые позволяют абстрагироваться от деталей хранения данных и сосредоточиться на методах их обработки.

Любую совокупность данных можно охарактеризовать двумя ключевыми понятиями, вокруг которых будет строиться практически всё дальнейшее обсуждение: *целостность* и *согласованность*, они же *integrity* и *consistency*. Общепринятого разделения между

ними нет, в связи с чем их часто путают. Концептуально можно сказать, что целостность включает в себя сугубо технический набор требований к данным в отдельности: например, доменные ограничения, а согласованность подразумевает ограничения на всю совокупность данных, соответствие которым не обязательно возможно проверить в отрыве от контекста. Можно подойти к вопросу более прагматично и сказать, что целостность включает такие ограничения, которые может проверить СУБД, а согласованность — такие, которые слишком тяжело описать декларативно, и их может проверить лишь человек. Понятно, что в любой из этих трактовок из согласованности следует целостность, но не наоборот. Иногда к этому ряду добавляют *непротиворечивость*, однако это скорее логический аспект согласованности, который не слишком интересен сам по себе.

Конечно, на несогласованные данные не может опираться ни один бизнес-процесс, поэтому наша конечная цель будет состоять прежде всего в том, чтобы поддерживать базу в валидном состоянии. Как станет ясно далее, это та ещё задача.

Архитектура данных

Уровни архитектуры

Система управления должна иметь полное представление о внутреннем устройстве базы данных. Какие сущности в ней выделяются, в какую структуру они объединяются, как с ней взаимодействовать, как сохранять её в памяти машины? — всё это положено знать любой СУБД, и всё это определяется *моделью данных*. Первоначально модели данных придумывались людьми в произвольном режиме, однако совсем скоро поняли, что это не совсем эффективно и что к вопросу следовало бы подойти системно; иначе говоря, архитектурно. Говоря ещё иначе, нужно было всё стандартизировать: возникла потребность в базовых принципах и простых моделях, на основе которых можно было бы создавать большие решения.

Занявшись этим вопросом, начали понимать, что проектирование базы данных на самом деле охватывает большее количество областей, чем можно было бы ожидать, и найти человека, который хорошо разбирался бы во всех из них — большая удача. А у специалистов разных сфер, в свою очередь, довольно разные взгляды на вопрос того, как должен работать продукт. Для того, чтобы как-то уладить этот вопрос, в 1975 году появился стандарт ANSI/SPARC, предлагающий три уровня архитектуры БД или, вернее говоря, три точки зрения на неё:

Внешний уровень	Чистая бизнес-логика: данные в нужном клиенту виде.
Концептуальный уровень	Формат представления данных со внешнего уровня.
Внутренний уровень	Полная абстракция от семантики: методы хранения, структуры данных, алгоритмы, шифрование, сжатие и проч.

Роль СУБД заключается в том, чтобы связать все из этих уровней воедино.

Уже поверх этих концепций выделяются конкретные модели данных, частично или полностью относящиеся к тем или иным уровням. Ключевых из них можно выделить три:

> ► **Модель «сущность — связь»**
> (на пересечении *внешнего* и *концептуального* уровней)
>
> Отражает наше реальное представление о предметной области: какие в ней представлены сущности, какими они наделены свойствами и как они связаны между собой.

> ► **Логическая модель**
> (на *концептуальном* уровне)
>
> Определяет, как представить модель «сущность — связь» в формальном виде. Непосредственно сущности и связи от неё не зависят; она задаёт лишь способ описания.

> ► **Физическая модель**
> (на пересечении *концептуального* и *внутреннего* уровней)
>
> Программное обеспечение. Определяет, как будет реализована логическая модель на машинном уровне.

Из них физическая модель нам наименее интересна; оставшиеся две же заслуживают отдельного обсуждения.

Модель «сущность — связь»

Концепция модели «сущность — связь», или же ER-модели (от *Entity-Relationship*), была предложена Питером Ченом в 1976 году; им же предложена и самая популярная графическая нотация для неё. Основные её составляющие уже упомянуты, однако о них стоит дополнительно сказать ещё несколько слов.

Под *сущностью* подразумевается понятие любого объекта из доменной области базы данных. В нотации Чена сущности обозначаются прямоугольниками.

Свойства, или же *атрибуты*, определяют характеристики этого объекта. Их можно категоризовать как простые или составные, обязательные или необязательные, однозначные или многозначные и.т.д.; причём соверешнно не всегда очевидно, к какой именно категории будет относиться то или иное свойство. Важно, что всякий атрибут характеризуется *доменом*, то есть областью допустимых значений. В нотации Чена атрибуты обозначаются овалами и связываются с сущностями при помощи линий.

Связь обозначает какую-либо зависимость между рядом сущностей. В зависимости от того, сколько сущностей объединяются одной связью, ей присваивается категория «один к одному», «один ко многим» или «многое ко многим». На основании наличия обязательных связей сами сущности также можно охарактеризовать как *сильные*, которые могут существовать без внешних зависимостей (например, человек), и *слабые*, для которых это неверно (например, паспорт у человека). В нотации Чена связи обозначаются ромбами.

#TODO kartinka

Принципиально важно помнить, что ER-модель не привязана ни к логической, ни тем более к физической модели: мы просто описываем реальность как она есть.

Логические модели данных

В отличие от ER-модели, логических моделей много, причем они действительно разные, ибо каждая из них по-своему решает компромиссы между целостностью и производительностью. В основе современных баз данных почти всегда лежит реляционная модель, которая в подробностях будет рассмотрена далее, однако перед этим важно оглянуться назад и посмотреть, как человечество к ней пришло.

Исконно одна из первых логических моделей — **иерархическая**. В сущности, иерархическая модель представляет собой лес, и если это может показаться странным выбором сейчас, то в 50-е годы прошлого века никто даже усомниться не мог в том, что это самый естественный способ представления данных. Деревьев тогда хватало: штатные структуры в кампаниях и государствах, сборочные производства, даже каталоги

со вложенными рубриками в библиотеках — и люди мыслили ими. Поэтому данные исконно было выбрано представлять в виде последовательности *полей*, — неделимых единиц, инкапсулирующих то или иное свойство сущности, — связанных указателями друг на друга и образующих ациклический граф.

#TODO картинка

Проблемы иерархической модели вытекают в первую очередь из того, какая структура данных лежит в её основе. Древовидная структура вынуждает дублировать данные: если несколько деталей используют одну и ту же гайку, то у каждой из них эта гайка всё равно должна быть в своём уникальном экземпляре, ибо в дереве у одной вершины не может быть два родителя (и, конечно, гайка никак не может быть родителем детали). Дублирование, как будет видно далее, очень негативно сказывается на целостности. Другая проблема в том, что дерево эффективно лишь тогда, когда мы спускаемся по нему, тогда как в обратном случае производительность довольно ощутимо падает. И, конечно, не всякая доменная область в принципе представима в иерархической модели.

В качестве альтернативы была выдвинута **сетевая** модель данных, единственное принципиальное отличие которой от иерархической в том, что она представляет собой не лес, а направленный ациклический граф.

#TODO картинка

Это сразу решает проблему избыточности и расширяет ряд областей, которые подвергаются моделированию, однако производительность становится ещё хуже; более того, семантическая сложность запроса в сетевой модели почти не влияет на время его выполнения. Уже в этот момент становится ясен главный краеугольный камень любой логической модели: нельзя повысить целостность, не потеряв в скорости работы, и наоборот.

В какой-то момент стало понятно, что пытаться улучшать иерархические или сетевые модели неэффективно (хотя попытки были), и нужно подходить к вопросу с принципиально другой перспективы. При этом нужда в модернизации действительно была, и основной недостаток актуального на тот момент стека был даже не в скорости работы и не в безопасности операций с точки зрения согласованности, а в том, что работать с ними было попросту тяжело. Графовые модели ещё называются *навигационными*, и это название как ничто другое отражает суть проблемы: запрос в такую базу неизбежно подразумевал ручную навигацию по ней, то есть указание полного пути к данным, из-за чего нередко приходилось писать целые программы только для того, чтобы получить конкретный кусочек информации.

Решением стала **реляционная** модель, которая сделала все связи *неявными*: они выводятся логически из значений конкретных свойств; например, студент связан с

группой тогда и только тогда, когда поле `groupId` у студента совпадает с полем `Id` у группы. Таким образом все пути полностью исключились из самой схемы: теперь они определяются автоматически в момент запроса. Это действительно кратно упрощает взаимодействие с базой, однако, как ни печально, в этом же кроется и главная проблема: если навигационные модели не допускали интерпретации, то реляционная обязывает пользователя самостоятельно наделять данные семантикой, в связи с чем мы теряем большое количество строгих гарантий относительно их состояния.

Этого, впрочем, ещё не было достаточно для того, чтобы добиться полной независимости от пути доступа: если наименьшей адресуемой единицей данных продолжит оставаться отдельная запись, то просмотр связей вида «один ко многим» всё ещё будет требовать указания точного порядка обхода. Наименьшей абстракцией, которая позволила бы сделать все запросы к такой базе однородными, является *множество*: действительно, если применять любой запрос к набору записей и получать в результате другой набор записей, то мы никогда не выйдем за пределы этого понятия. В реляционной модели множество записей называется *отношением* (*от англ. relation*); отсюда и её название. Ставя это понятие во главу угла, мы делаем последний шаг к нашей исходной цели — полному отделению данных от способа их хранения.

#TODO картинк

На этом, впрочем, эволюция логических моделей не остановилась. После реляционной модели появилась *постреляционная*, которая разрешила использовать в качестве значений полей вложенные структуры данных, в то время как реляционная строго требовала их атомарности. По сей день существуют вариации на тему *многомерной* модели, практически кэширующей данные в гиперкубы (OLAP-кубы), которые можно рассекать и довольно быстро получать сложные срезы данных, однако такие модели страдают от несогласованности: кэш имеет свойство устаревать. Предпринимались и попытки перенести на базы данных объекто-ориентированный подход, результаты которых на практике применяются лишь для довольно специфических задач.

Реляционные базы данных

Ключевые понятия

Отношение

У истоков реляционной модели стоял британский учёный *Эдгар Франк Кодд*, работавший в исследовательском центре IBM на момент публикации в 1970 году первой официальной работы по этой теме, которая регулярно здесь цитируется. К этому моменту он трудился

над ней уже почти десять лет и продолжил заниматься ей и далее, в том числе и вне IBM. В 1981 году он получил за свою разработку премию Тьюринга.

Кодд был хорошим математиком, поэтому мог позволить себе заложить в модель строгую математическую основу. Это послужило большим преимуществом, ибо формальный фундамент позволял оценивать две главные вещи: время работы и риски для согласованности.

Конечно, в математике нельзя сказать «отношение — это таблица», потому что понятия таблицы, в общем-то, нет, да и не всякая таблица является отношением. Кодд использует этот термин в общепринятом математическом смысле:

> **Info**
>
> **Отношением** R для заданных множеств $S_1, \ldots, S_n$ является любое подмножество их декартова произведения.

Практически отношение является множеством из кортежей $D = \langle d_1, \ldots, d_n \rangle$ таких, что для всякого элемента всякого кортежа верно $d_i \in S_i$; это значит, что все «столбцы» хранят данные одинакового типа. d_i здесь считается *атрибутом*, а S_i — его *доменом*, то есть его областью допустимых значений. Примечательно, что для Кодда не было важно, а можно ли явно построить такое множество или существует ли вообще предикат, который позволяет проверить принадлежность элемента ему; он использовал это понятие в самом широком смысле.

Слово «кортеж» изначально также использовалось в привычном смысле, то есть как название для упорядоченного набора элементов фиксированной длины. Порядок внутри кортежа нужен был для того, чтобы отличать друг от друга различные атрибуты, соответствующие одинаковым доменам. Позже стало понятно, что атрибуты гораздо практичнее определять по имени, а не по позиции, в связи с чем изменилось и понятие кортежа: теперь его следовало воспринимать как отображение из множества имён в множество значений (или, что то же самое, как множество пар «имя — значение»), и порядок элементов внутри него перестал иметь значение. Конечно, имена атрибутов, в отличие от доменов, обязаны быть уникальными.

Число n атрибутов в отношении называется его *степенью*, а число m кортежей в нём — его *кардинальностью*. Множество всех отношений называется *реляционной схемой*; отношения также наделяются именем.

#TODO картинк

Уже в самòм этом определении отражены шесть главных свойств, отличающих отношение от произвольного множества. Говоря конкретнее,

I. Каждое поле содержит ровно одно неделимое значение.

II. Каждый кортеж уникален.

III. Имя отношения в реляционной схеме уникально.

IV. Имя атрибута в отношении уникально.

V. Значения атрибута принадлежат одному и тому же домену.

VI. Порядок следования кортежей и атрибутов не имеет значения.

Они кажутся очень простыми и естественными до тех пор, пока мы остаёмся в пределах математической теории. На практике же, как будет видно далее, почти всех из них в той или иной степени нарушаются, причем повсеместно.

Неизвестные значения атрибутов в отношении могут помечаться маркером NULL. Это можно формализовать различными способами: например, можно сказать, что отношение может содержать переменные и в такое случае отражает не единственное состояние системы, а все возможные. Для Кодда было принципиально важно одно: каждый отдельно взятый NULL встречается в отношении не более чем единожды, то есть, в частности, он не может быть выражен через специально выделенное значение, потому что это вообще не значение. Как ни печально, на практике это не достигается. Введение такого маркера вынужденно расширяет логику операций; в частности, любое сравнение с NULL возвращает результат «неизвестно», а любая арифметическая операция с ним возвращает его же.

Ключи

Ключи используются для идентификации кортежей и построения связей. Формально,

Определение

Суперключ — множество атрибутов внутри отношения, единственным образом идентифицирующее кортеж.

Из определения отношения понятно, что у каждого кортежа есть по меньшей мере один суперключ — он сам.

Определение

Потенциальный ключ — суперключ, не содержащий собственных подмножеств, являющихся суперключами.

Иными словами, потенциальный ключ — *минимальный* набор атрибутов, достаточный для идентификации объекта. При этом он не обязан быть единственным:

#TODO картинк про потенциальный клю

В связи с этим вводится понятие *первичного ключа*:

Определение

Первичный ключ — потенциальный ключ, который был однозначно выбран для идентификации кортежей.

Связь между двумя кортежами из разных отношений осуществляется при помощи *внешних ключей*:

Определение

Внешний ключ — множество атрибутов внутри отношения, соответствующее потенциальному ключу некоторого, может быть того же самого, отношения.

Это проясняет концепцию неявных связей. Ключи не закладываются в модель на уровне путей, а лишь определяют, как отличить одну сущность от другой, и этого оказывается достаточно для того, чтобы пользователь мог самостоятельно определить, существует ли между ними связь. Минимальность первичного и потенциального ключей при этом гарантирует стабильность при изменении описательных атрибутов.

Такая модель особенно удобна, когда используются *естественные* ключи, то есть состоящие из полей, несущих содержательную информацию об объекте. В противовес им выделяются *суррогатные* ключи на основе специально созданных служебных полей, служащих только для уникальной идентификации кортежа.

С понятием ключа связано два вида целостности, специфичных для реляционной модели.

Сущностная целостность говорит о том, что ни один атрибут первичного ключа не может иметь значение NULL. Её нарушение означает, что мы не можем гарантированно отличить один кортеж от другого, а значит и строить связи, ибо они тесно завязаны на уникальности ключей.

Ссылочная целостность говорит о том, что значение внешнего ключа должно либо полностью соответствовать значению потенциального ключа, либо полностью состоять из NULL-значений. Её нарушение означает, что какая-то ссылка указывает на несуществующий кортеж, и точно так же не позволяет нам нормально строить связи.

Реляционная алгебра

Кодд понимал, что СУБД поверх его модели будут писаться совершенно по-разному, и в конечном итоге всё это с лёгкостью могло бы стать несовместимо между собой. Для того, чтобы этого избежать, нужно было строго описать не только саму основу в виде отношения, но и то, как проводить над ними операции.

Для этого выделялось два пути: *алгебра*, то есть замкнутая система операций, и *исчисление*, то есть формальный язык для вывода утверждений (практически — логика первого порядка). Теорема Кодда говорит о том, что оба из этих подходов равносильны по выразительности, поэтому на практике они используются гибридно. Исчисление больше подходит для клиентской стороны, ибо позволяет описать, что нужно получить, а не как это получить; алгебра же, наоборот, описывает механизм, и в связи с этим является идеальным фундаметом для вычислений. Поскольку описательная часть не слишком интересна, стоит подробнее остановиться именно на вычислительной.

Основными из интересующих нас операций являются:

> I. **Проекция** $\Pi_{a_1,\ldots,a_n}(R)$
>
> Определяет новое отношение, которое содержит вертикальное подмножество R и создаётся посредством извлечения значений указанных атрибутов и исключения из результата строк-дубликатов.

II. **Выборка** $\Pi_{a_1,\ldots,a_n}(R)$

Определяет новое отношение, которое содержит горизонтальное подмножество R и создаётся посредством извлечения значений кортежей, удовлетворяющих некоторому предикату.

III. **Объединение** $R \cup S$

Определяет новое отношение, которое включает:

- все кортежи, содержащиеся только в R;
- все кортежи, содержащиеся только в S;
- кортежи, содержащиеся одновременно и в R, и в S, с исключением дубликатов.

Не для любых двух отношений существует их объединение. Из этих соображений был введён критерий *совместимости по объединению*, который подразумевает, что отношения состоят из одинакового количества атрибутов, и эти атрибуты попарно имеют общие домены. При этом важно помнить про то, что формально одинаковые домены не обязаны быть семантически одинаковыми, так что объединение позволяет наделать глупостей.

IV. **Разность** $R - S$

Определяет новое отношение, состоящее из кортежей, которые содержатся в отношении R, но отсутствуют в отношении S.

Для существования разности кортежи также обязаны быть совместимы по объединению.

V. **Пересечение** $R \cap S$

Определяет новое отношение, состоящее из кортежей, которые содержатся и в отношении R, и в отношении S.

Для существования пересечения кортежи обязаны быть совместимы по объединению.

VI. **Декартово произведение** $R \times S$

Определяет новое отношение, являющееся результатом конкатенации каждого кортежа из R с каждым кортежем из S.

VII. **Тета-соединение** $R \bowtie_F S$

Определяет новое отношение, содержащее горизонтальное подмножество декартова произведения двух отношений, к которому применена выборка по предикату вида

$$F = R.a_i \Theta S.b_i$$

где $\Theta \in \{=, \neq, >, <, \ldots\}$; иными словами, F представляет собой попарное сравнение значений двух выбранных атрибутов для всех кортежей.

Если тета-соединение проводится по предикату равенства, оно называется *эквисоединением*.

VIII. **Естественное соединение** $R \bowtie S$

Эквисоединение двух отношений, выполненное по всем общим атрибутам, из результатов которого исключается по одному экземпляру каждого общего атрибута.

IX. **Левое внешнее соединение** $R \bowtie S$

Естественное соединение, при котором в результирующее отношение включаются также кортежи отношения R, не имеющие совпадений в общих столбцах отношения S.

#TODO картинк наверно

Главное, чего хотелось от алгебры — это чтобы после применения операции к изначально осмысленным отношениям на выходе получалось осмысленное отношение. Если мы получим бессмысленный результат и сохраним его в базу, то согласованность нарушится почти гарантированно. Основной риск здесь, конечно, в операции тета-соединения, которая позволяет объединять кортежи вне зависимости от семантики атрибутов. Кодд пытался решить это, введя операцию естественного соединения, однако в общем случае для двух разных отношений мы просто не можем гарантировать, что два одноимённых атрибута с одинаковыми доменами — это одно и то же семантическое свойство сущности. В связи с этим естественное соединение в принципе не подвергается безопасной реализации, а это значит, что бремя следования здравому смыслу всецело лежит на пользователе.

Выкладки из SQL

SQL (от англ. *Structured Query Language* — язык структурированных запросов) — декларативный язык программирования, разработанный под эгидой IBM по мотивам реляционной модели. Словосочетание «по мотивам» здесь подчёркивает тот факт, что, по большому счёту, SQL не является истинно реляционным языком, на что неоднократно указывали Кодд и Дейт. Так, например,

- поле строкового типа в SQL не атомарно, так как поддерживает сопоставление шаблонов;

- результаты выборок в SQL допускают дублирующиеся результаты;
- NULL в SQL реализован как отдельное значение;
- в SQL важен порядок следования атрибутов, и к ним можно обращаться по номеру.

Впрочем, следует понимать, что всё это — инженерные компромиссы, неизбежные или экономически выгодные, и они не делают язык непригодным к использованию; более того, некоторые решения SQL, которые расходятся с решениями авторов реляционной модели, могут показаться даже более естественными.

Декларативность языка неизбежно следует из философии самой модели: он подразумевался максимально простым (хотя позже настолько усложнился, что в итоге всё равно стал практически инструментом для программиста, а не для пользователя) и максимально отделённым от физического представления данных. Сами же запросы можно реализовывать по-разному, и, конечно, потребителя никто не хотел напрягать этой работой: от него требовалось лишь описать, что он хочет получить.

Прежде всего нас интересуют *операторы манипуляции данными*. В стандартном SQL их четыре: `SELECT`, `INSERT`, `UPDATE` и `DELETE`. Их внутреннее устройство довольно похоже, поэтому мы рассмотрим в подробности только первый из них. Итак, структура оператора проекции следующая:

```
SELECT [DISTINCT|ALL] {*|[ColumnExpression [as NewName]] [,...]}
FROM Table1 [as NewName]
[{INNER|LEFT OUTER|FULL} JOIN Table2 [as NewName] on Condition][...]
[WHERE Condition]
[GROUP BY ColumnList][HAVING Condition]
[ORDER BY ColumnList [ASC|DESC]]
```

Здесь:

`SELECT ... FROM Table1 [as NewName]`	Запись операции проекции на отношении `Table1`.
`[DISTINCT|ALL]`	Параметр, определяющий, будет ли результирующее отношение содержать дубликаты; понятно, что исходно реляционная модель вообще не подразумевает такой опции.

`{*|[ColumnExpression [as NewName]] [,...]}`	Список проецируемых или вычисляемых атрибутов.
`[{INNER|LEFT OUTER|FULL} JOIN Table2 ... on Condition]`	Запись операции тета-соединения.
`[WHERE Condition]`	Запись операции выборки.
`[GROUP BY ColumnList][HAVING Condition]`	Запись оператора *группировки/агрегации*, которая пришел в виде расширения к классической алгебре.
`[ORDER BY ColumnList [ASC|DESC]]`	Запись оператора сортировки.

Порядок выполнения этого запроса в полной форме следующий:

1. `FROM JOIN ON` — создали результирующее отношение;
2. `WHERE` — выбрали удовлетворяющие предикату кортежи;
3. `GROUP BY HAVING` — сгруппировали кортежи;
4. `SELECT` — спроецировали атрибуты (до группировки включительно нам могли понадобиться все из них, после — уже нет)
5. `DISTINCT` — удалили дубликаты;
6. `ORDER BY` — отсортировали результат.

Самая дорогостоящая операция здесь — создание таблицы или, говоря точнее, непосредственно `JOIN`. Конечно, реализация тета-соединения оставляет довольно много простора для воображения: его можно производить наивно, а можно, например, предварительно отсортировать отношения и линейно проитерироваться по соответствующим атрибутам при помощи двух указателей, если при этом соединение происходит по равенству (это называется *слиянием*); можно отказаться от сравнений вообще и использовать хэш-таблицы в попытке приблизиться к линейной оценке времени; можно использовать подзапросы вместо явного соединения, чтобы оптимизировать расход памяти. Так или иначе, объём данных, с которыми приходится работать при выполнении этой операции, неизбежно огромен, а используют её практически в любом запросе. Это следует держать в голове к моменту, когда далее зайдёт разговор об оптимизациях.

Нормализация

В результате размышлений на тему причин нарушений согласованности выяснилось, что одну из них можно устранить ещё на уровне проектирования. Эта проблема заключается в структурной *избыточности* данных, то есть в бессодержательном дублировании. Понятно, что мы не можем не дублировать данные вообще: например, группу так или иначе необходимо хранить для каждого студента, и если мы откажемся от этого, то потеряем ценную информацию. Направление же достаточно хранить для каждой группы в частности, а хранение этих данных для студента — пример избыточности.

В частности, избыточность является причиной нарушений, или *аномалий*, трёх видов:

> ► **Аномалия модификации**
>
> Изменение значения одной записи повлечёт за собой просмотр всей таблицы и изменение некоторых других записей. Например, если мы храним направление в таблице со студентами, то перевод одной из групп на новое направление приведет к тому, что эту информацию необходимо обновить для всех, кто учится в этой группе.

> ► **Аномалия удаления**
>
> При удалении одной записи может пропасть независимая информация. Например, если мы храним данные клиента в таблице с заказами, то при удалении заказа клиент перестаёт существовать.

> ► **Аномалия вставки**
>
> Информацию в таблицу нельзя поместить, пока она неполна, или для этого требуется дополнительный просмотр таблицы. Например, в предыдущем примере мы не сможем записать информацию о клиенте без заказов; если же мы попытаемся записать туда фиктивные «пустые» значения, то при следующем обновлении мы будем вынуждены проверить, не записали ли мы ранее этого клиента.

С аномалиями необходимо бороться. Для этого Кодд ввёл понятие *нормальной формы*, подразумевающее такую декомпозицию отношений, при которой условная степень избыточности не превышает определённого порядка.

Границы действия нормальных форм будут определяться в понятиях *зависимостей* атрибутов:

> ⓘ **Определение**
>
> В отношении R атрибут X **функционально зависит** от атрибута Y (обозначается $X \Longrightarrow Y$), если каждому значению X соответствует в точности одно значение Y.

Атрибут X называется *детерминантом*, а атрибут Y — *зависимой частью*.

Конечная цель рассматриваемых далее нормальных форм в том, чтобы все детерминанты стали первичными ключами.

Нам будут важны преимущественно три конкретных категории функциональной зависимости. Будем называть ФЗ:

- *частичной*, если она подразумевает зависимость атрибута от части составного потенциального ключа;
- *полной*, если она подразумевает зависимость атрибута от всего составного потенциального ключа;
- *транзитивной*, если существует такой атрибут Z, что $X \implies Z$ и $Z \implies Y$.

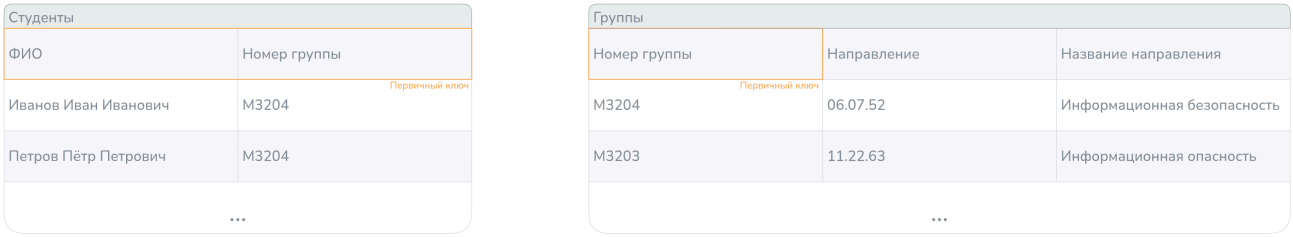
Итак, первые пять нормальных форм формулируются следующим образом:

> **► Первая нормальная форма**
>
> Отношение находится в 1НФ, если все его атрибуты являются простыми.
>
> Это минимальное требование для того, чтобы можно было вообще говорить о декомпозиции: если поле, например, содержит под собой JSON, то мы не можем наверняка утверждать, что данные внутри него не избыточны.

> **► Вторая нормальная форма**
>
> Отношение находится в 2НФ, если оно находится в 1НФ и все его неключевые атрибуты функционально полно зависят от потенциального ключа. Если в отношении ключ несоставной, то это условие выполнено по умолчанию, и во многом именно с этим связано использование суррогатных ключей.
>
> Эта форма устраняет аномалии, связанные с частичными функциональными зависимостями. Рассмотрим таблицу «Студенты», которая содержит информацию о группе и направлении:
>
> Студенты
>
> | ФИО | Номер группы | Направление | Название направления |
> |---|---|---|---|
> | | Первичный ключ | | |
> | Иванов Иван Иванович | М3204 | 06.07.52 | Информационная безопасность |
> | Петров Пётр Петрович | М3204 | 06.07.52 | Информационная безопасность |
> | ... | | | |
>
> Первичным ключом здесь будет сочетание ФИО и номера группы, ибо мы полагаем, что если полные тёзки и есть, то точно не в одной группе. Направление будет зависеть

лишь от номера группы, то есть от части составного ключа, из-за чего это отношение не находится в 2НФ. Оно подвержено аномалии модификации (перенесли группу на другое направление), вставки (нельзя создать направление без студентов) и удаления (информация о направлении теряется, если на нём нет студентов).

Его можно привести к 2НФ, разбив на два:



Видно, что ни одна из этих конкретных аномалий не встречается в получившихся отношениях.

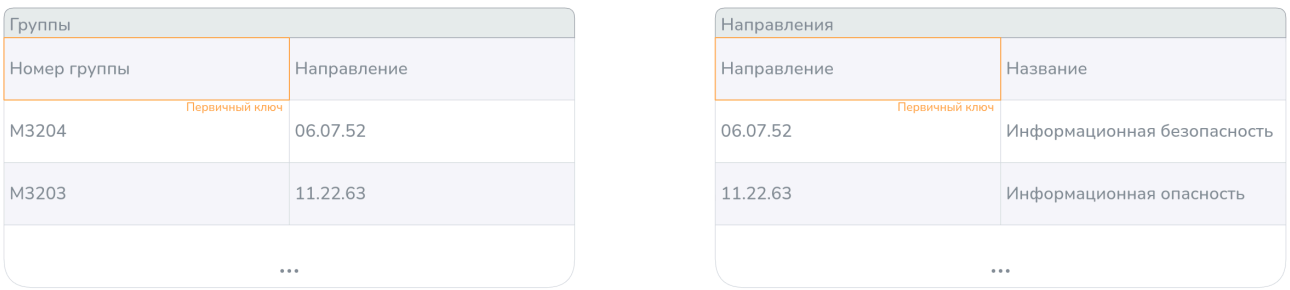
► Третья нормальная форма

Отношение находится в 3НФ, если оно находится в 2НФ и:

- все неключевые атрибуты попарно взаимно независимы или, что то же самое,
- не один неключевой атрибут не находится в транзитивной зависимости от потенциального ключа.

Эта форма устраняет аномалии, связанные с транзитивными зависимостями. Рассмотрим таблицу «Группы» из предыдущего примера. Она находится в 2НФ, однако название направления в ней зависит от номера направления или, в соответствии со вторым определением 3НФ, транзитивно зависит от номера группы через направление. Оно подвержено аномалии модификации (изменили название направления), вставки (нельзя создать направление без групп) и удаления (информация о названии направления теряется, если на нём нет групп).

Его можно привести к 3НФ, разбив на два:



Видно, что в этих отношениях в принципе нет аномалий.

► Нормальная форма Бойса — Кодда

Отношение находится в БКНФ, если оно находится в 3НФ и детерминанты всех зависимостей являются потенциальными ключами.

Эта форма устраняет неочевидные аномалии, упущенные в 3НФ. Рассмотрим более сложный пример с отношением «Проектная деятельность», содержащим информацию о номере проекта, ИСУ участника, его паспортных данных и его роли:

Проектная деятельность

ИСУ	Номер проекта	Паспортные данные	Роль
	Потенциальный ключ	Потенциальный ключ	
123456	1	6752-239239	Научный руководитель
239239	2	1234-567890	Научный руководитель
123456	2	6752-239239	Исследователь
...			

Потенциальных ключа здесь два, ибо как ИСУ, так и паспортные данные однозначно определяют личность. Это отношение находится в 3НФ, так как единственное неключевое поле «Роль» полно функционально зависит от любого из ключей. Тем не менее, она не находится в БКНФ, так как паспортные данные функционально зависят от ИСУ, которое не является потенциальным ключом. Оно подвержено в первую очередь аномалии модификации (изменим паспортные данные у участника).

Его можно привести к БКНФ, разбив на два:

Проектная деятельность

ИСУ	Номер проекта	Роль
	Первичный ключ	
123456	1	Научный руководитель
239239	2	Научный руководитель
123456	2	Исследователь
...		

Паспортные данные

ИСУ	Паспортные данные
Первичный ключ	
123456	6752-239239
239239	1234-567890
123456	6752-239239
...	

Видно, что в этих отношениях в принципе нет аномалий.

► Четвёртая нормальная форма

Отношение находится в 4НФ, если оно находится в БКНФ и не содержит нетривиальных многозначных зависимостей.

Рассмотрим отношение «Нагрузка», содержащего информацию о дисциплинах, лекторах и практиках:

Нагрузка

Дисциплина	Лектор	Практик
		Первичный ключ
Математический анализ	Михайлов К. Н.	Мандаринова Е. А.
Технологии ИИ в разработке ПО	Иванов И. И,	Петров И. И.
Математический анализ	Михайлов К. Н.	Лимонова А. Е.
...		

Все три поля образуют первичный ключ, ибо на одной дисциплине может быть более одного лектора, у лектора может быть более одной дисциплины и более одного практика, а у практика может быть более одного лектора.

Может показаться, что мы избыточно дублируем пару «дисциплина — лектор», и следует декомпозировать отношение на «дисциплина — лектор» и «лектор — практик», однако это верно ровно до тех пор, пока мы не рассматриваем совместимость конкретного лектора с конкретным практиком.

Этот пример показывает, что старшие нормальные формы решают проблемы, которые, в общем-то, не обязаны ими являться, и в их случае ответ на вопрос «а является ли это поведение вообще аномалией?» варьируется в зависимости от бизнес-процесса. В промышленных базах данных практически никогда не используются формы старше БКНФ.

Основная проблема глубоко декомпозированных систем в том, что расстояние между связанными блоками данных в них может быть достаточно велико. Если раньше вся информация о студенте сразу находилась в составе одного отношения, то теперь это отношение приходится каждый раз строить заново, объединяя его части. В связи с этим иногда имеет место *денормализация*: намеренный отказ от соблюдения нормальных форм в угоду производительности.

#TODO про Дейта

Источники

1. Маятин А. В. — Лекции по основам баз данных, осенний семестр, 2026
2. Википедия — База данных (https://ru.wikipedia.org/wiki/База_данных)
3. Википедия — ER-модель (https://ru.wikipedia.org/wiki/ER-модель)

4. IBM History — The relational database (https://www.ibm.com/history/relational-database)
5. E. F. Codd — A Relational Model of Data for Large Shared Data Banks (https://www.engineering.upenn.edu/~zives/03f/cis550/codd.pdf)
6. Э. Ф. Кодд — Реляционная модель данных для больших совместно используемых банков данных (перевод М. Р. Когаловского) (https://smtp.citforum.ru/database/classics/codd/)
7. Wikipedia — Codd's Theorem (https://en.wikipedia.org/wiki/Codd's_theorem)
8. Википедя — SQL (https://ru.wikipedia.org/wiki/SQL)